

Heterogeneous Programming

Introduction to OpenCL

Juan C. Pichel

Centro Singular de Investigación en Tecnoloxías Intelixentes

Universidade de Santiago de Compostela (USC)

Máster en HPC

citius.gal

- **Juan Carlos Pichel** (juancarlos.pichel@usc.es)
- Office 111 (CiTIUS - USC)
- OpenCL: 6 hours
- Grading: Homework (up to 1.5 points, 15% PAH grade)

Many slides included in this course are from:

Hands on OpenCL - Simon McIntosh-Smith and Tom Deakin



Ravishekhar Banger and Koushik Bhattacharyya.

OpenCL Programming by Example.

Packt Publishing, 2013.



Benedict Gaster, Lee Howes, David R. Kaeli, Perhaad Mistry, and Dana Schaa.

Heterogeneous Computing with OpenCL.

Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1st edition, 2011.



David R. Kaeli, Perhaad Mistry, Dana Schaa, and Dong Ping Zhang.

Heterogeneous Computing with OpenCL 2.0.

Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1st edition, 2015.



Aaftab Munshi, Benedict Gaster, Timothy G. Mattson, James Fung, and Dan Ginsburg.

OpenCL Programming Guide.

Addison-Wesley Professional, 1st edition, 2011.

Table of contents

- 1 An Overview of OpenCL
- 2 Important OpenCL concepts
- 3 Overview OpenCL APIs
- 4 Kernel Programming
- 5 Memory Hierarchy
- 6 Synchronization
- 7 Heterogeneous Computing



Table of contents

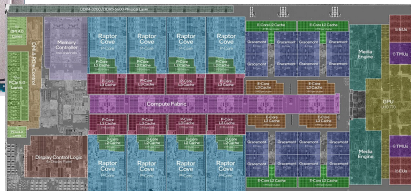
- 1 An Overview of OpenCL
- 2 Important OpenCL concepts
- 3 Overview OpenCL APIs
- 4 Kernel Programming
- 5 Memory Hierarchy
- 6 Synchronization
- 7 Heterogeneous Computing



It's a Heterogeneous world

A modern computing platform includes:

- One or more CPUs
- One or more GPUs
- DSP processors
- Accelerators
- ... other?

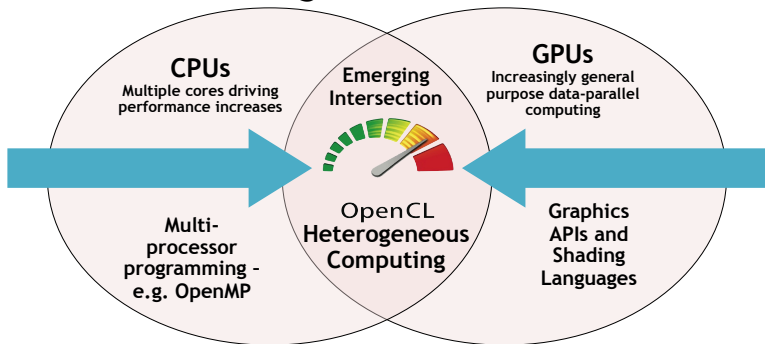


E.g. Intel® Raptor Lake:

- Up to 8 P-core and 16 E-core, UHD770 GPU with 32 execution units

OpenCL lets Programmers write a single portable program that uses ALL resources in the heterogeneous platform

Industry Standards for Programming Heterogeneous Platforms



OpenCL - Open Computing Language

Open, royalty-free standard for portable, parallel programming of heterogeneous parallel computing CPUs, GPUs, and other processors

Programming and Runtime Framework for Application Acceleration

Offload compute-intensive kernels onto parallel heterogeneous processors
 CPUs, GPUs, DSPs, FPGAs, Tensor Processors
 OpenCL C or C++ kernel languages

Platform Layer API

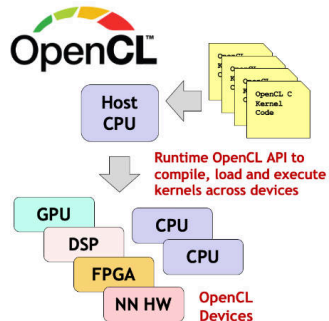
Query, select and initialize compute devices

Runtime API

Build and execute kernels programs on multiple devices

Explicit Application Control

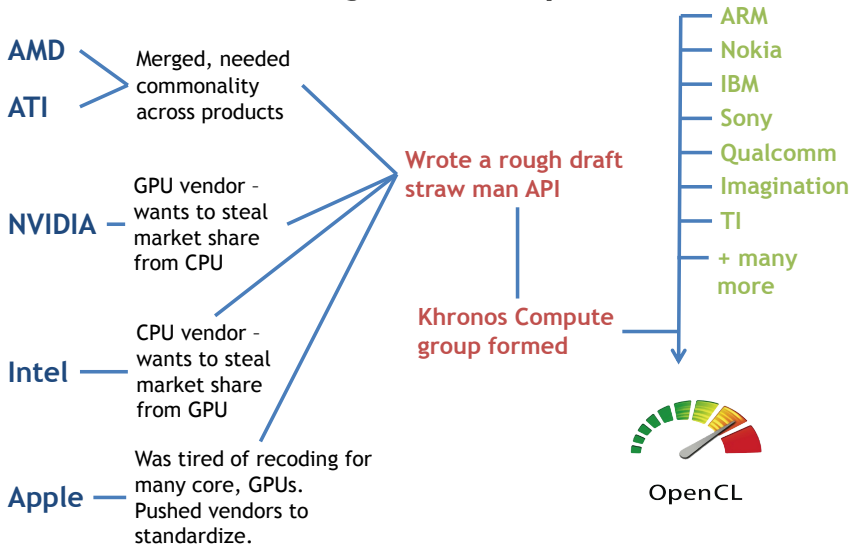
Which programs execute on what device
 Where data is stored in memories in the system
 When programs are run, and what operations are dependent on earlier operations



Complements GPU-only APIs

Simpler programming model
 Relatively lightweight run-time
 More language flexibility, e.g., pointers
 Rigorously defined numeric precision

The origins of OpenCL



Third party names are the property of their owners.

OpenCL is Widely Deployed and Used

Desktop Creative Apps



Parallel Languages



Machine Learning Libraries and Frameworks



The industry's most pervasive, cross-vendor, open standard for low-level heterogeneous parallel programming

Molecular Modelling Libraries



Machine Learning Compilers



Vision and Imaging Libraries



Math and Physics Libraries



Accelerated Implementations

https://en.wikipedia.org/wiki/List_of_OpenCL_applications

This work is licensed under a Creative Commons Attribution 4.0 International License

© The Khronos® Group Inc. 2020 - Page 4

OpenCL 3.0



Single Source C++ Programming
Full support for features in C++14-based Kernel Language



API and Language Specs
Brings C++14-based Kernel Language into core specification



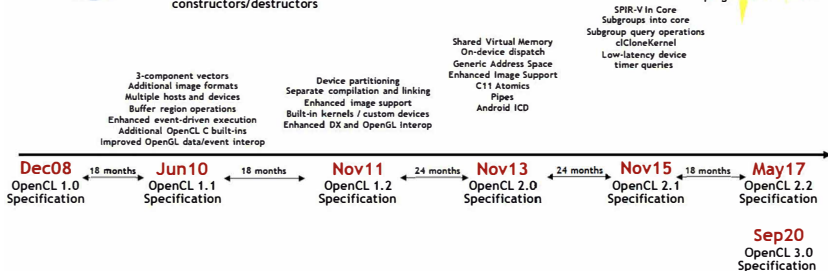
Portable Kernel Intermediate Language
Support for C++14-based kernel language e.g. constructors/destructors

OpenCL C++ Kernel Language
SPIR-V 1.2 with C++ support
SYCL 2.2 single source C++
Pipes

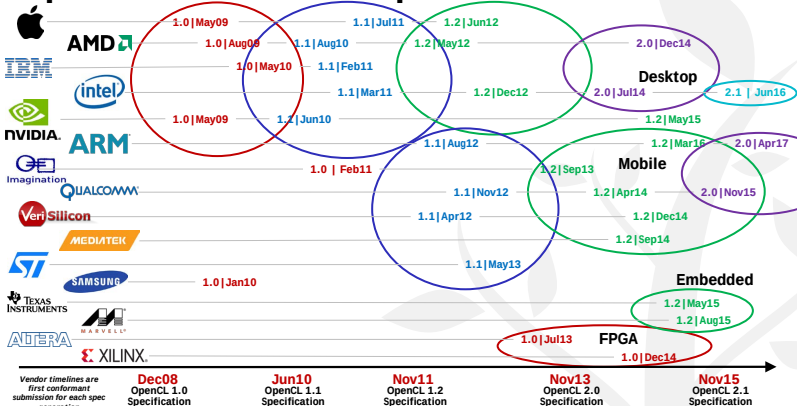
Efficient device-scope communication between kernels

Code Gen Optimizations:

- Specialization constants at SPIR-V compilation time
- Constructors and destructors of program scope global objects
- User callbacks can be set at program release time

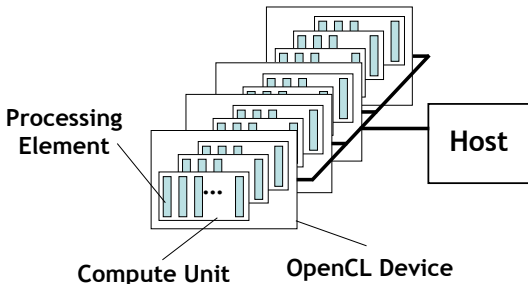


OpenCL Conformant Implementations



© Copyright Khronos Group 2017 - Page 8

OpenCL Platform Model



- One *Host* and one or more *OpenCL Devices*
 - Each OpenCL Device is composed of one or more *Compute Units*
 - Each Compute Unit is divided into one or more *Processing Elements*
- Memory divided into *host memory* and *device memory*

OpenCL Platform Example

(One node, two CPU sockets, two GPUs)

CPU:

- Treated as one OpenCL device
 - One CU per core
 - 1 PE per CU, or if PEs mapped to SIMD lanes, n PEs per CU, where n matches the SIMD width
- Remember:
 - the CPU will also have to be its own host!

GPUs:

- Each GPU is a separate OpenCL device
- Can use CPU and all GPU devices concurrently through OpenCL

CU = Compute Unit; PE = Processing Element

An Overview of OpenCL

Setting up the OpenCL environment on your computer

- **macOS**: OpenCL works out of the box! Just compile your programs with: `-framework OpenCL`
- **Intel CPUs and GPUs**: Linux and Windows drivers. Follow the instructions in:
<https://software.intel.com/en-us/articles/opencl-drivers>
- **nVIDIA GPUs**: OpenCL support is included in the latest NVIDIA GPU drivers. In addition, install CUDA toolkit.

An Overview of OpenCL

Setting up the OpenCL environment on CESGA FT2

- Download the OpenCL API 1.2 Reference Guide from:
<https://www.khronos.org/files/opencv-1-2-quick-reference-card.pdf>
- Connect to FT3 or log into <https://portalusuarios.cesga.es> using your CESGA account.
- Open a terminal and execute:

```
$> wget https://persoal.citius.usc.es/jcpichel/docs/  
Miniconda3-latest-Linux-x86_64.sh  
$> chmod 744 Miniconda3-latest-Linux-x86_64.sh  
$> ./Miniconda3-latest-Linux-x86_64.sh  
$> conda config --add channels conda-forge  
$> conda install pyopencl pocl pocl-cuda ocl-icd opencl-headers
```

An Overview of OpenCL

Before starting the labs...

- Import the OpenCL Labs to your account (pah-openc1-labs folder):

```
$> module load pocl/1.6—CUDA—system  
$> wget https://persoal.citius.usc.es/jcpichel/docs/pah-openc1-labs.zip  
$> unzip pah-openc1-labs.zip  
$> cd pah-openc1-labs/Exercises  
$> make
```

- Only CPUs are detected. To use GPUs in FT3 (number of users at the same time is restricted):

```
$> compute —gpu  
$> module load pocl/1.6—CUDA—system
```

- Or send GPU jobs to the queue (use scripts in the repo)

An Overview of OpenCL

Exercise 1: Platform Information (~10 minutes)

- **Goal:**
Verify that you can run a simple OpenCL program.
- **Procedure:**
Take the provided DeviceInfo program, inspect it in the editor of your choice, build the program and run it.
- **Output:**
Information about the installed OpenCL platforms and the devices visible to them.

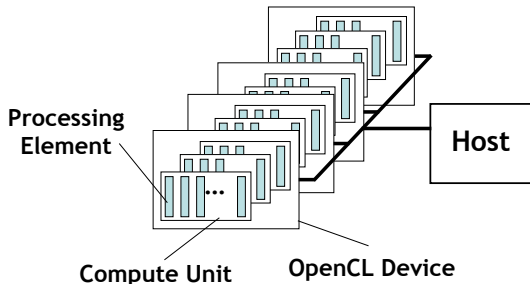
[illegible]

Table of contents

- 1 An Overview of OpenCL
- 2 Important OpenCL concepts
- 3 Overview OpenCL APIs
- 4 Kernel Programming
- 5 Memory Hierarchy
- 6 Synchronization
- 7 Heterogeneous Computing



OpenCL Platform Model



- One *Host* and one or more *OpenCL Devices*
 - Each OpenCL Device is composed of one or more *Compute Units*
 - Each Compute Unit is divided into one or more *Processing Elements*
- Memory divided into *host memory* and *device memory*

The **BIG** idea behind OpenCL

- Replace loops with functions (a **kernel**) executing at each point in a problem domain
 - E.g., process a 1024x1024 image with one kernel invocation per pixel or $1024 \times 1024 = 1,048,576$ kernel executions

Traditional loops

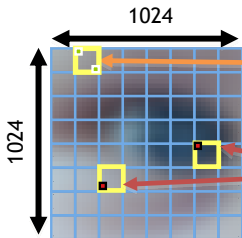
```
void
mul(const int n,
    const float *a,
    const float *b,
    float *c)
{
    int i;
    for (i = 0; i < n; i++)
        c[i] = a[i] * b[i];
}
```

Data Parallel OpenCL

```
__kernel void
mul(__global const float *a,
    __global const float *b,
    __global float *c)
{
    int id = get_global_id(0);
    c[id] = a[id] * b[id];
}
// many instances of the kernel,
// called work-items, execute
// in parallel
```

An N-dimensional domain of work-items

- **Global** Dimensions:
 - 1024x1024 (whole problem space)
- **Local** Dimensions:
 - 64x64 (**work-group**, executes together)



Synchronization between
work-items possible only
within **work-groups**:
barriers and **memory fences**

Cannot synchronize
between **work-groups**
within a kernel

- Choose the dimensions that are “best” for your algorithm

OpenCL N Dimensional Range (NDRange)

- The problem we want to compute should have some **dimensionality**;
 - For example, compute a kernel on all points in a cube
- When we execute the kernel we specify **up to 3 dimensions**
- We also **specify the total problem size** in each dimension - this is called the **global size**
- We associate each point in the iteration space with a **work-item**



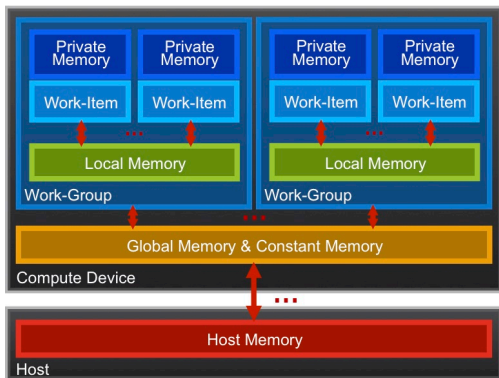
OpenCL N Dimensional Range (NDRange)

- Work-items are grouped into **work-groups**; work-items within a work-group can share **local memory** and can **synchronize**
- We can specify the number of work-items in a work-group - this is called the **local** (work-group) size
- Or the OpenCL run-time can choose the work-group size for you (usually not optimally)



OpenCL Memory model

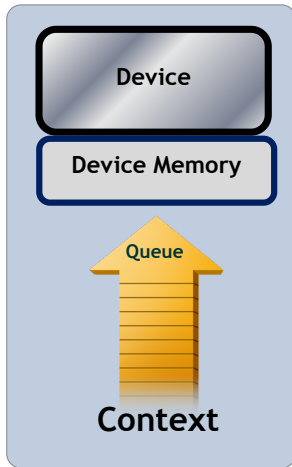
- **Private Memory**
 - Per work-item
- **Local Memory**
 - Shared within a work-group
- **Global Memory / Constant Memory**
 - Visible to all work-groups
- **Host memory**
 - On the CPU



Memory management is **explicit**:
You are responsible for moving data from
host → global → local *and* back

Context and Command-Queues

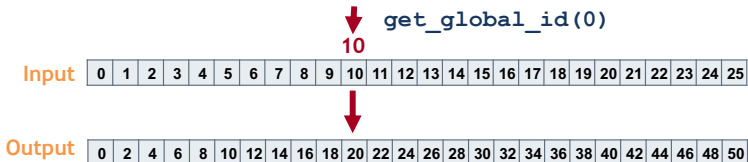
- **Context:**
 - The environment within which kernels execute and in which synchronization and memory management is defined.
- The **context** includes:
 - One or more devices
 - Device memory
 - One or more command-queues
- All **commands** for a device (kernel execution, synchronization, and memory transfer operations) are submitted through a **command-queue**.
- Each **command-queue** points to a single device within a context.



Execution model (kernels)

- OpenCL execution model ... define a problem domain and execute an instance of a **kernel** for each point in the domain

```
__kernel void times_two(
    __global float* input,
    __global float* output)
{
    int i = get_global_id(0);
    output[i] = 2.0f * input[i];
}
```



Building Program Objects

- The program object encapsulates:
 - A context
 - The program kernel source or binary
 - List of target devices and build options
- The C API build process to create a program object:
 - `clCreateProgramWithSource()`
 - `clCreateProgramWithBinary()`

OpenCL uses **runtime compilation** ... because in general you don't know the details of the target device when you ship the program

```
__kernel void
horizontal_reflect(read_only image2d_t src,
                  write_only image2d_t dst)
{
    int x = get_global_id(0); // x-coord
    int y = get_global_id(1); // y-coord
    int width = get_image_width(src);
    float4 src_val = read_imagef(src, sampler,
                                (int2)(width-1-x, y));
    write_imagef(dst, (int2)(x, y), src_val);
}
```

Compile for
GPU

GPU
code

Compile for
CPU

CPU
code

Example: vector addition

- The “hello world” program of data parallel programming is a program to add two vectors

$C[i] = A[i] + B[i] \text{ for } i=0 \text{ to } N-1$

- For the OpenCL solution, there are two parts
 - Kernel code
 - Host code

Vector Addition - Kernel

```
__kernel void vadd(__global const float *a,  
                  __global const float *b,  
                  __global float *c)  
{  
    int gid = get_global_id(0);  
    c[gid] = a[gid] + b[gid];  
}
```



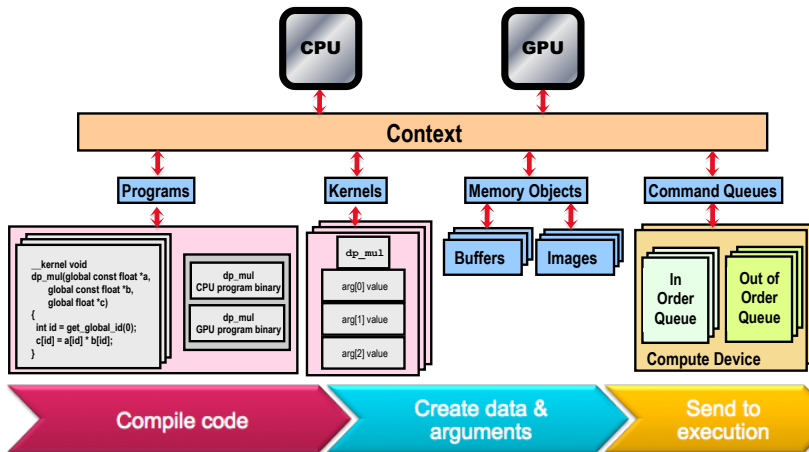
Vector Addition - Host

- The host program is the code that runs on the host to:
 - Setup the environment for the OpenCL program
 - Create and manage kernels
- 5 simple steps in a basic host program:
 1. Define the **platform** ... platform = devices+context+queues
 2. Create and Build the **program** (dynamic library for kernels)
 3. Setup **memory** objects
 4. Define the **kernel** (attach arguments to kernel functions)
 5. Submit **commands** ... transfer memory objects and execute kernels



As we go over the next set of slides, cross reference content on the slides to the reference card. This will help you get used to the reference card and how to pull information from the card and express it in code.

The basic platform and runtime APIs in OpenCL (using C)



1. Define the platform

- Grab the first available **platform**:

```
err = clGetPlatformIDs(1, &firstPlatformId,  
                        &numPlatforms);
```

- Use the first CPU **device** the platform provides:

```
err = clGetDeviceIDs(firstPlatformId,  
                     CL_DEVICE_TYPE_CPU, 1, &device_id, NULL);
```

- Create a simple **context** with a single device:

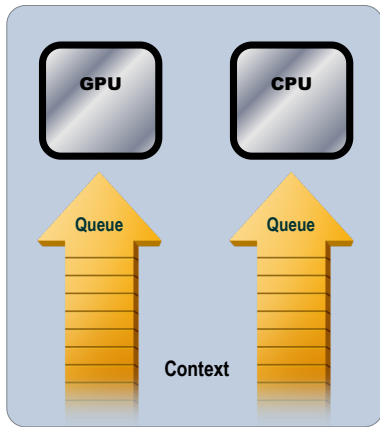
```
context = clCreateContext(firstPlatformId, 1,  
                          &device_id, NULL, NULL, &err);
```

- Create a simple **command-queue** to feed our device:

```
commands = clCreateCommandQueue(context, device_id,  
                                 0, &err);
```

Command-Queues

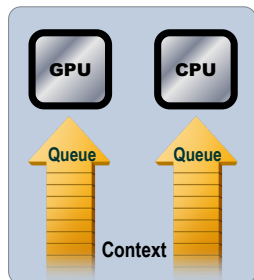
- Commands include:
 - Kernel executions
 - Memory object management
 - Synchronization
- The only way to submit **commands** to a device is through a **command-queue**.
- Each command-queue points to a **single** device within a context.
- **Multiple command-queues can feed a single device.**
 - Used to define independent streams of commands that don't require synchronization



Command-Queue execution details

Command queues can be configured in different ways to control how commands execute

- **In-order queues:**
 - Commands are enqueued and complete in the order they appear in the program (program-order)
- **Out-of-order queues:**
 - Commands are enqueued in program-order but can execute (and hence complete) in any order.
- **Execution of commands in the command-queue are guaranteed to be completed at synchronization points**
 - Discussed later



2. Create and Build the program

- Define source code for the kernel-program as a string literal (great for toy programs) or read from a file (for real applications).
- Build the **program object**:

```
program = clCreateProgramWithSource(context, 1  
                                     (const char**) &KernelSource, NULL, &err);
```

- **Compile** the program to create a “dynamic library” from which specific kernels can be pulled:

```
err = clBuildProgram(program, 0, NULL, NULL, NULL, NULL);
```


Error messages

- Fetch and print **error** messages:

```
if (err != CL_SUCCESS) {  
    size_t len;  
    char buffer[2048];  
    clGetProgramBuildInfo(program, device_id,  
        CL_PROGRAM_BUILD_LOG, sizeof(buffer), buffer, &len);  
    printf("%s\n", buffer);  
}
```

- Important to do check all your OpenCL API error messages!
- Easier in C++ with try/catch (see later)

3. Setup Memory Objects

- For vector addition we need 3 memory objects, one each for input vectors A and B, and one for the output vector C.
- Create input vectors and assign values **on the host**:

```
float h_a[LENGTH], h_b[LENGTH], h_c[LENGTH];  
for (i = 0; i < length; i++) {  
    h_a[i] = rand() / (float)RAND_MAX;  
    h_b[i] = rand() / (float)RAND_MAX;  
}
```

- Define **OpenCL** memory objects:

```
d_a = clCreateBuffer(context, CL_MEM_READ_ONLY,  
                      sizeof(float)*count, NULL, NULL);  
d_b = clCreateBuffer(context, CL_MEM_READ_ONLY,  
                      sizeof(float)*count, NULL, NULL);  
d_c = clCreateBuffer(context, CL_MEM_WRITE_ONLY,  
                      sizeof(float)*count, NULL, NULL);
```

What do we put in device memory?

Memory Objects:

- A handle to a reference-counted region of **global** memory.

There are two kinds of memory object

- **Buffer** object:
 - Defines a linear collection of bytes (*“just a C array”*).
 - The contents of buffer objects are fully exposed within kernels and can be accessed using pointers
- **Image** object:
 - Defines a two- or three-dimensional region of memory.
 - Image data can **only** be accessed with read and write functions, i.e. these are opaque data structures. The read functions use a sampler.

Used when interfacing with a graphics API such as OpenGL. We won't use image objects in this tutorial.

Creating and manipulating buffers

- Buffers are declared on the host as type: `cl_mem`
- Arrays in host memory hold your original host-side data:

```
float h_a[LENGTH], h_b[LENGTH];
```

- Create the `buffer` (`d_a`), assign `sizeof(float)*count` bytes from “`h_a`” to the buffer and copy it into device memory:

```
cl_mem d_a = clCreateBuffer(context,  
    CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR,  
    sizeof(float)*count, h_a, NULL);
```

Conventions for naming buffers

- It can get confusing about whether a host variable is just a regular C array or an OpenCL buffer
- A useful convention is to prefix the names of your regular **h**ost C arrays with “**h_**” and your OpenCL buffers which will live on the **d**evice with “**d_**”



Creating and manipulating buffers

- Other common **memory flags** include:
`CL_MEM_WRITE_ONLY`, `CL_MEM_READ_WRITE`
- These are from the point of view of the **device**
- Submit command to copy the buffer back to host memory at “h_c”:
 - `CL_TRUE` = blocking, `CL_FALSE` = non-blocking

```
clEnqueueReadBuffer(queue, d_c, CL_TRUE,  
                    sizeof(float)*count, h_c,  
                    NULL, NULL, NULL);
```

4. Define the kernel

- Create **kernel object** from the **kernel function** “vadd”:

```
kernel = clCreateKernel(program, "vadd", &err);
```

- Attach arguments of the kernel function “vadd” to memory objects:

```
err = clSetKernelArg(kernel, 0, sizeof(cl_mem), &d_a);  
err |= clSetKernelArg(kernel, 1, sizeof(cl_mem), &d_b);  
err |= clSetKernelArg(kernel, 2, sizeof(cl_mem), &d_c);  
err |= clSetKernelArg(kernel, 3, sizeof(unsigned int),  
                        &count);
```

5. Enqueue commands

- Write **Buffers** from host into **global** memory (as **non-blocking** operations):

```
err = clEnqueueWriteBuffer(commands, d_a, CL_FALSE,  
                           0, sizeof(float)*count, h_a, 0, NULL, NULL);  
err = clEnqueueWriteBuffer(commands, d_b, CL_FALSE,  
                           0, sizeof(float)*count, h_b, 0, NULL, NULL)
```

- Enqueue the kernel for execution (note: in-order so OK):

```
err = clEnqueueNDRangeKernel(commands, kernel, 1,  
                              NULL, &global, &local, 0, NULL, NULL);
```


5. Enqueue commands

- Read back result (as a blocking operation). We have an in-order queue which assures the previous commands are completed before the read can begin.

```
err = clEnqueueReadBuffer(commands, d_c, CL_TRUE,  
    sizeof(float)*count, h_c, 0, NULL, NULL);
```

Vector Addition - Host Program

```
// create the OpenCL context on a GPU device
cl_context context = clCreateContextFromType(0,
      CL_DEVICE_TYPE_GPU, NULL, NULL, NULL);

// get the list of GPU devices associated with context
clGetContextInfo(context, CL_CONTEXT_DEVICES, 0, NULL, &cb);

cl_device_id[] devices = malloc(cb);
clGetContextInfo(context, CL_CONTEXT_DEVICES, cb, devices, NULL);

// create a command-queue
cmd_queue = clCreateCommandQueue(context, devices[0], 0, NULL);

// allocate the buffer memory objects
memobjs[0] = clCreateBuffer(context, CL_MEM_READ_ONLY |
      CL_MEM_COPY_HOST_PTR, sizeof(cl_float)*n, srcA, NULL);
memobjs[1] = clCreateBuffer(context, CL_MEM_READ_ONLY |
      CL_MEM_COPY_HOST_PTR, sizeof(cl_float)*n, srcB, NULL);

memobjs[2] = clCreateBuffer(context, CL_MEM_WRITE_ONLY,
      sizeof(cl_float)*n, NULL, NULL);

// create the program
program = clCreateProgramWithSource(context, 1,
      &program_source, NULL, NULL);
```

```
// build the program
err = clBuildProgram(program, 0, NULL, NULL, NULL, NULL);

// create the kernel
kernel = clCreateKernel(program, "vec_add", NULL);

// set the args values
err = clSetKernelArg(kernel, 0, (void *) &memobjs[0],
      sizeof(cl_mem));
err |= clSetKernelArg(kernel, 1, (void *) &memobjs[1],
      sizeof(cl_mem));
err |= clSetKernelArg(kernel, 2, (void *) &memobjs[2],
      sizeof(cl_mem));

// set work-item dimensions
global_work_size[0] = n;

// execute kernel
err = clEnqueueNDRangeKernel(cmd_queue, kernel, 1, NULL,
      global_work_size, NULL, 0, NULL, NULL);

// read output array
err = clEnqueueReadBuffer(cmd_queue, memobjs[2],
      CL_TRUE, 0,
      n*sizeof(cl_float), dst,
      0, NULL, NULL);
```

Vector Addition - Host Program

```
// create the OpenCL context on a GPU device
cl_context context = clCreateContextFromType(0,
                                             CL_DEVICE_TYPE_GPU, NULL, NULL, NULL);
```

Define platform and queues

```
// get the list of GPU devices associated with context
clGetContextInfo(context, CL_CONTEXT_DEVICES, 0, NULL, &cb);

cl_device_id devices[10];
clGetContextInfo(context, CL_CONTEXT_DEVICES, cb, devices, NULL);
```

```
// create a command-queue
cmd_queue = clCreateCommandQueue(context, devices[0], 0, NULL);
```

```
// allocate the buffer memory objects
memobjs[0] = clCreateBuffer(context, CL_MEM_READ_ONLY |
                             CL_MEM_COPY_HOST_PTR, sizeof(cl_float)*n, srcA, NULL);
memobjs[1] = clCreateBuffer(context, CL_MEM_READ_ONLY, sizeof(cl_float)*n, NULL, NULL);
```

Define memory objects

```
memobjs[2] = clCreateBuffer(context, CL_MEM_WRITE_ONLY,
                             sizeof(cl_float)*n, NULL, NULL);
```

```
// create the program
program = clCreateProgramWithSource(context, 1,
                                     &srcA, NULL, NULL);
```

Create the program

```
// build the program
err = clBuildProgram(program, 0, NULL, NULL, NULL);
```

Build the program

```
// create the kernel
kernel = clCreateKernel(program, "vec_add", NULL);
```

```
// set the args values
err = clSetKernelArg(kernel, 0, (void *) &memobjs[0],
```

Create and setup kernel

```
err |= clSetKernelArg(kernel, 1, sizeof(cl_mem) * sizeof(memobjs[1]),
err |= clSetKernelArg(kernel, 2, (void *) &memobjs[2],
                        sizeof(cl_mem));
```

```
// set work-item dimensions
global_work_size[0] = n;
```

```
// execute kernel
err = clEnqueueTask(cmd_queue, kernel, 1, NULL, 0, NULL, NULL);
```

Execute the kernel

```
// read output array
err = clEnqueueReadBuffer(cmd_queue, memobjs[2], 0, 0, n, dst,
```

Read results on the host

It's complicated, but most of this is “boilerplate” and not as bad as it looks.

Table of contents

- 1 An Overview of OpenCL
- 2 Important OpenCL concepts
- 3 Overview OpenCL APIs**
- 4 Kernel Programming
- 5 Memory Hierarchy
- 6 Synchronization
- 7 Heterogeneous Computing



Host programs can be “ugly”

- OpenCL’s goal is extreme portability, so it exposes *everything*
 - (i.e. it is quite verbose!).
- But most of the host code is the same from one application to the next - the re-use makes the verbosity a non-issue.
- You can package common API combinations into functions or even C++ or Python classes to make the reuse more convenient.

The C++ Interface

- Khronos has defined a common C++ header file containing a high level interface to OpenCL, **cl.hpp**
- This interface is dramatically easier to work with¹
- Key features:
 - Uses common defaults for the platform and command-queue, saving the programmer from extra coding for the most common use cases
 - Simplifies the basic API by bundling key parameters with the objects rather than requiring verbose and repetitive argument lists
 - Ability to “call” a kernel from the host, like a regular function
 - Error checking can be performed with C++ exceptions

¹ especially for C++ programmers...

C++ Interface: setting up the host program

- Enable OpenCL API **Exceptions**. Do this **before** including the header file

```
#define __CL_ENABLE_EXCEPTIONS
```

- Include key header files ... both standard and custom

```
#include <CL/cl.hpp>    // Khronos C++ Wrapper API
#include <cstdio>        // For C style
#include <iostream>      // For C++ style IO
#include <vector>        // For C++ vector types
```

For information about C++, see
the appendix:
“C++ for C programmers”.

C++ interface: The vadd host program

```

std::vector<float>
    h_a(N), h_b(N), h_c(N);
// initialize host vectors...

cl::Buffer d_a, d_b, d_c;

cl::Context context(
    CL_DEVICE_TYPE_DEFAULT);

cl::CommandQueue
    queue(context);

cl::Program program(
    context,
    loadprogram("vadd.cl"),
    true);

// Create the kernel functor
cl::make_kernel<cl::Buffer,
    cl::Buffer, cl::Buffer, int>
    vadd(program, "vadd");

// Create buffers
// True indicates CL_MEM_READ_ONLY
// False indicates CL_MEM_READ_WRITE

d_a = cl::Buffer(context,
    h_a.begin(), h_a.end(), true);

d_b = cl::Buffer(context,
    h_b.begin(), h_b.end(), true);

d_c = cl::Buffer(context,
    CL_MEM_READ_WRITE,
    sizeof(float) * LENGTH);

// Enqueue the kernel
vadd(cl::EnqueueArgs(
    queue,
    cl::NDRange(count)),
    d_a, d_b, d_c, count);

cl::copy(queue,
    d_c, h_c.begin(), h_c.end());

```

The C++ Buffer Constructor

- This is the API definition:
 - `Buffer(startIterator, endIterator, bool readOnly, bool useHostPtr)`
- The `readOnly` boolean specifies whether the memory is `CL_MEM_READ_ONLY` (true) or `CL_MEM_READ_WRITE` (false)
 - You must specify a true or false here
- The `useHostPtr` boolean is default false
 - Therefore the array defined by the iterators is **implicitly copied** into device memory
 - If you specify **true**:
 - The memory specified by the iterators must be **contiguous**
 - The context **uses the pointer** to the host memory, which becomes device accessible - this is the same as `CL_MEM_USE_HOST_PTR`
 - The array **is not** copied to device memory
- We can also specify a context to use as the first argument in this API call

The C++ Buffer Constructor

- When using the buffer constructor which uses C++ vector iterators, remember:
 - This is a blocking call
 - The constructor will enqueue a copy to the first Device in the context (when useHostPtr == false)
 - The OpenCL runtime will **automatically** ensure the buffer is copied across to the actual device you enqueue a kernel on later if you enqueue the kernel on a different device within this context

The Python Interface

- A python library by Andreas Klockner from University of Illinois at Urbana-Champaign
- This interface is dramatically easier to work with¹
- Key features:
 - Helper functions to choose platform/device at runtime
 - `getInfo()` methods are class attributes - no need to call the method itself
 - Call a kernel as a method
 - Multi-line strings - no need to escape new lines!

¹ not just for python programmers...

Setting up the host program

- Import the pyopencl library

```
import pyopencl as cl
```

- Import numpy to use arrays etc.

```
import numpy
```

- Some of the examples use a helper library to print out some information

```
import deviceinfo
```



```
N = 1024

# create context, queue and program
context = cl.create_some_context()
queue = cl.CommandQueue(context)
kernelsource = open('vadd.cl').read()
program = cl.Program(context, kernelsource).build()

# create host arrays
h_a = numpy.random.rand(N).astype(float32)
h_b = numpy.random.rand(N).astype(float32)
h_c = numpy.empty(N).astype(float32)

# create device buffers
mf = cl.mem_flags
d_a = cl.Buffer(context, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=h_a)
d_b = cl.Buffer(context, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=h_b)
d_c = cl.Buffer(context, mf.WRITE_ONLY, h_c.nbytes)

# run kernel
vadd = program.vadd
vadd.set_scalar_arg_dtypes([None, None, None, numpy.uint32])
vadd(queue, h_a.shape, None, d_a, d_b, d_c, N)

# return results
cl.enqueue_copy(queue, h_c, d_c)
```


■ **Goal:**

■ **Procedure:**

[illegible]

■ **Output:**

A message verifying that the chain of vector additions completed successfully.

Table of contents

- 1 An Overview of OpenCL
- 2 Important OpenCL concepts
- 3 Overview OpenCL APIs
- 4 Kernel Programming**
- 5 Memory Hierarchy
- 6 Synchronization
- 7 Heterogeneous Computing



OpenCL C for Compute Kernels

- Derived from **ISO C99**
 - A few **restrictions**: no recursion, function pointers, functions in C99 standard headers ...
 - Preprocessing directives defined by C99 are supported (`#include` etc.)
- Built-in data types
 - Scalar and vector data types, pointers
 - Data-type conversion functions:
 - `convert_type<_sat><_roundingmode>`
 - Image types:
 - `image2d_t`, `image3d_t` and `sampler_t`



OpenCL C for Compute Kernels

- Built-in functions — *mandatory*
 - Work-Item functions, math.h, read and write image
 - Relational, geometric functions, synchronization functions
 - printf (v1.2 only, so not currently for NVIDIA GPUs)
- Built-in functions — *optional* (called “extensions”)
 - Double precision, atomics to global and local memory
 - Selection of rounding mode, writes to image3d_t surface

OpenCL C Language Highlights

- Function qualifiers
 - **__kernel** qualifier declares a function as a kernel
 - I.e. makes it visible to host code so it can be enqueued
 - Kernels can call other kernel-side functions
- Address space qualifiers
 - **__global**, **__local**, **__constant**, **__private**
 - Pointer kernel arguments must be declared with an address space qualifier
- Work-item functions
 - **get_work_dim()**, **get_global_id()**, **get_local_id()**, **get_group_id()**
- Synchronization functions
 - **Barriers** - all work-items within a work-group must execute the barrier function before any work-item can continue
 - **Memory fences** - provides ordering between memory operations

OpenCL C Language Restrictions

- Pointers to functions are **not** allowed
- Pointers to pointers allowed **within** a kernel, but not as an argument to a kernel invocation
- Bit-fields are not supported
- Variable length arrays and structures are not supported
- Recursion is not supported (yet!)
- Double types are **optional** in OpenCL v1.1, but the key word is reserved
(note: most implementations support double)

Worked example: Linear Algebra

- **Definition:**

- The branch of mathematics concerned with the study of vectors, vector spaces, linear transformations and systems of linear equations.

- **Example:** Consider the following system of linear equations

$$x + 2y + z = 1$$

$$x + 3y + 3z = 2$$

$$x + y + 4z = 6$$

- This system can be represented in terms of vectors and a matrix as the classic “ $Ax = b$ ” problem.

$$\begin{pmatrix} 1 & 2 & 1 \\ 1 & 3 & 3 \\ 1 & 1 & 4 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 6 \end{pmatrix}$$

Solving $Ax=b$

- LU Decomposition:
 - transform a matrix into the product of a lower triangular and upper triangular matrix. It is used to solve a linear system of equations.

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & -1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 5 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 1 \\ 1 & 3 & 3 \\ 1 & 2 & 4 \end{pmatrix}$$

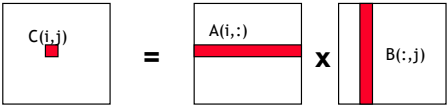
- We solve for x , given a problem $Ax=b$
 - $Ax=b$ $LUx=b$
 - $Ux=(L^{-1})b$ $x = (U^{-1})(L^{-1})b$

So we need to be able to do matrix multiplication

Matrix multiplication: sequential code

We calculate $C=AB$, where all three matrices are $N \times N$

```
void mat_mul(int N, float *A, float *B, float *C)
{
    int i, j, k;
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            C[i*N+j] = 0.0f;
            for (k = 0; k < N; k++) {
                // C(i, j) = sum(over k) A(i,k) * B(k,j)
                C[i*N+j] += A[i*N+k] * B[k*N+j];
            }
        }
    }
}
```



Dot product of a row of A and a column of B for each element of C

Matrix multiplication performance

- Serial C code on CPU (single core).

Case	MFLOPS	
	CPU	GPU
Sequential C (not OpenCL)	887.2	N/A

Device is Intel® Xeon® CPU, E5649 @ 2.53GHz
using the gcc compiler.

These are not official benchmark results. You may observe completely different results should you run these tests on your own system.

Third party names are the property of their owners.

Matrix multiplication: sequential code

```
void mat_mul(int N, float *A, float *B, float *C)
{
    int i, j, k;
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            C[i*N+j] = 0.0f;
            for (k = 0; k < N; k++) {
                // C(i, j) = sum(over k) A(i,k) * B(k,j)
                C[i*N+j] += A[i*N+k] * B[k*N+j];
            }
        }
    }
}
```

We turn this into an OpenCL kernel!

Matrix multiplication: OpenCL kernel (1/2)

```
__kernel void mat_mul(  
    const int N,  
    __global float *A, __global float *B, __global float *C)  
{  
    int i, j, k;  
    for (i = 0; i < N; i++) {  
        for (j = 0; j < N; j++) {  
            // C(i, j) = sum(over k) A(i,k) * B(k,j)  
            for (k = 0; k < N; k++) {  
                C[i*N+j] += A[i*N+k] * B[k*N+j];  
            }  
        }  
    }  
}
```

Mark as a kernel function and
specify memory qualifiers

Matrix multiplication: OpenCL kernel (2/2)

```
__kernel void mat_mul(  
    const int N,  
    __global float *A, __global float *B, __global float *C)  
{  
    int i, j, k;  
    i = get_global_id(0);  
    j = get_global_id(1);  
    for (k = 0; k < N; k++) {  
        // C(i, j) = sum(over k) A(i,k) * B(k,j)  
        C[i*N+j] += A[i*N+k] * B[k*N+j];  
    }  
}
```

Remove outer loops and set
work-item co-ordinates

Matrix multiplication: OpenCL kernel

```
__kernel void mat_mul(  
    const int N,  
    __global float *A, __global float *B, __global float *C)  
{  
    int i, j, k;  
    i = get_global_id(0);  
    j = get_global_id(1);  
    // C(i, j) = sum(over k) A(i,k) * B(k,j)  
    for (k = 0; k < N; k++) {  
        C[i*N+j] += A[i*N+k] * B[k*N+j];  
    }  
}
```

Matrix multiplication: OpenCL kernel improved

Rearrange and use a local scalar for intermediate C element values (a common optimization in Matrix Multiplication functions)

```
__kernel void mmul(  
    const int N,  
    __global float *A,  
    __global float *B,  
    __global float *C)  
{  
    int k;  
    int i = get_global_id(0);  
    int j = get_global_id(1);  
    float tmp = 0.0f;  
    for (k = 0; k < N; k++)  
        tmp += A[i*N+k]*B[k*N+j];  
    C[i*N+j] += tmp;  
}
```

Matrix multiplication host program (C++ API)

```
int main(int argc, char *argv[])
{
    std::vector<float> h_A, h_B, h_C; // matrices
    int Mdim, Ndim, Pdim; // A[N][P], B[P][M], C[N][M]
    int i, err;
    // ... s in each matrix
    double start; // timing data
    cl::Program;

    Ndim = Pdim * Mdim;
    szA = Ndim * Pdim;
    szB = Pdim * Mdim;
    szC = Ndim * Mdim;
    h_A = std::vector<float>(szA);
    h_B = std::vector<float>(szB);
    h_C = std::vector<float>(szC);

    initmat(Mdim, Ndim, Pdim, h_A, h_B, h_C);

    // Compile for first kernel to setup program
    program = cl::Program::Source, true);
    Context context(device_infos);
    cl::CommandQueue queue(context, device_infos);
    std::vector<cl::Device> devices;
    context.getDevices(CL_DEVICE_TYPES);
    cl::Device device = devices[0];
    std::string s = device.getInfo<CL_DEVICE_NAME>();
    std::cout << "Using OpenCL Device "
    << s << "\n";
}
```

Declare and initialize data

Setup the platform and build program

```
// Set
// and
initmat
cl::Buffer
cl::Buffer
cl::Buffer

CL_MEM_WRITE_ONLY,
sizeof(float) * szC);
```

Setup buffers and write A and B matrices to the device memory

```
cl::make_kernel<int, int, int,
cl::Buffer, cl::Buffer, cl::Buffer>
Zero_mat(Ndim, Mdim, h_C);
start_time = wtime();
```

Create the kernel functor

```
naive(cl::EnqueueArgs(queue,
cl::NDRange(Ndim, Mdim)),
Ndim, Mdim, Pdim, h_A, h_B, h_C);
cl::copy(h_A, h_B, h_C);
end();
run_time = wtime() - start_time;
results(Mdim, Ndim, Pdim, h_C, run_time);
```

Run the kernel and collect results

Note: To use the default context/queue/device, skip this section and remove the references to context, queue and device.

Matrix multiplication performance

- Matrices are stored in global memory.

Case	MFLOPS	
	CPU	GPU
Sequential C (not OpenCL)	887.2	N/A
C(i,j) per work-item, all global	3,926.1	3,720.9

Device is Tesla® M2090 GPU from NVIDIA® with a max of 16 compute units, 512 PEs
Device is Intel® Xeon® CPU, E5649 @ 2.53GHz

These are not official benchmark results. You may observe completely different results should you run these tests on your own system.

Third party names are the property of their owners.

Kernel Programming

Exercise 5: Matrix Multiplication (~20 minutes)

■ **Goal:**

To write your first complete OpenCL kernel “from scratch”. To multiply a pair of matrices.

■ **Procedure:**

Start with the provided matrix multiplication OpenCL host program. Create a kernel to do the multiplication. Modify the provided OpenCL host program to use your kernel. Verify the results.

■ **Output:**

A message verifying the code has completed successfully. Report the runtime and the MFLOPS.



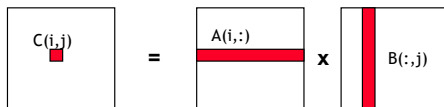
Table of contents

- 1 An Overview of OpenCL
- 2 Important OpenCL concepts
- 3 Overview OpenCL APIs
- 4 Kernel Programming
- 5 Memory Hierarchy**
- 6 Synchronization
- 7 Heterogeneous Computing



Optimizing matrix multiplication

- MM cost determined by FLOPS and memory movement:
 - $2 \cdot n^3 = O(n^3)$ FLOPS
 - Operates on $3 \cdot n^2 = O(n^2)$ numbers
- To optimize matrix multiplication, we must ensure that for every memory access we execute as many FLOPS as possible.
- Outer product algorithms are faster, but for pedagogical reasons, let's stick to the simple dot-product algorithm.

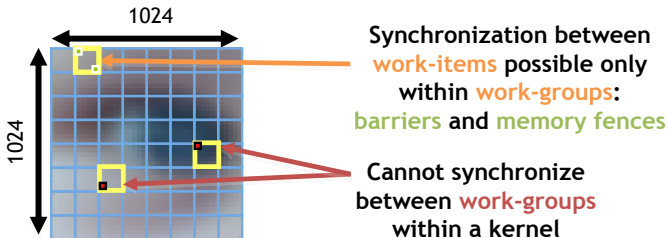


Dot product of a row of A and a column of B for each element of C

- We will work with work-item/work-group sizes and the memory model to optimize matrix multiplication

An N-dimensional domain of work-items

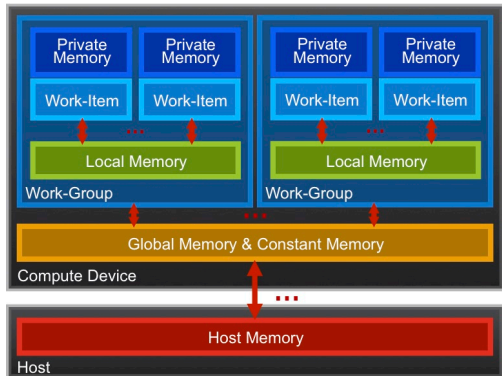
- **Global** Dimensions:
 - 1024x1024 (whole problem space)
- **Local** Dimensions:
 - 128x128 (**work-group**, executes together)



- Choose the dimensions that are “best” for your algorithm

OpenCL Memory model

- **Private Memory**
 - Per work-item
- **Local Memory**
 - Shared within a work-group
- **Global/Constant Memory**
 - Visible to all work-groups
- **Host memory**
 - On the CPU

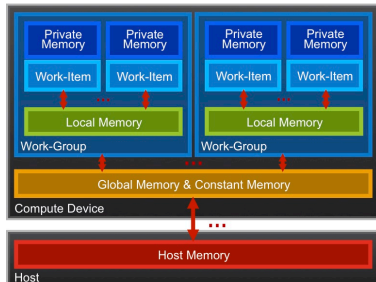


Memory management is **explicit**:

You are responsible for moving data from
host → global → local *and* back

OpenCL Memory model

- **Private Memory**
 - Fastest & smallest: $O(10)$ words/WI
- **Local Memory**
 - Shared by all WI's in a work-group
 - But not shared between work-groups!
 - $O(1-10)$ Kbytes per work-group
- **Global/Constant Memory**
 - $O(1-10)$ Gbytes of Global memory
 - $O(10-100)$ Kbytes of Constant memory
- **Host memory**
 - On the CPU - GBytes



Memory management is **explicit**:

$O(1-10)$ Gbytes/s bandwidth to discrete GPUs for
Host $\leftrightarrow$ Global transfers

Private Memory

- Managing the memory hierarchy is one of the most important things to get right to achieve good performance
- Private Memory:
 - A **very scarce** resource, only a few tens of 32-bit words per Work-Item at most
 - If you use **too much** it **spills to global memory** or **reduces the number of Work-Items** that can be run at the same time, potentially harming performance*
 - Think of these like registers on the CPU

* Occupancy on a GPU

Local Memory*

- Tens of KBytes per Compute Unit
 - As multiple Work-Groups will be running on each CU, this means only a fraction of the total Local Memory size is available to each Work-Group
- Assume $O(1-10)$ KBytes of Local Memory per Work-Group
 - Your kernels are responsible for transferring data between Local and Global/Constant memories ... there are optimized library functions to help
 - E.g. `async_work_group_copy()`,
`async_workgroup_strided_copy()`, ...
- Use Local Memory to hold data that can be **reused by all the work-items** in a work-group
- Access patterns to Local Memory affect performance in a similar way to accessing Global Memory
 - Have to think about things like coalescence & bank conflicts

* Typical figures for a 2013 GPU

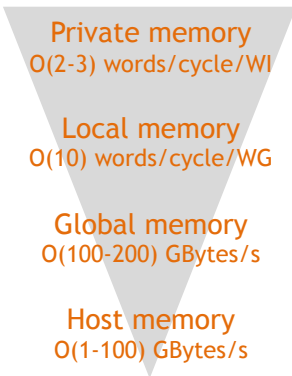
Local Memory

- **Local Memory** doesn't always help...
 - CPUs don't have special hardware for it
 - This can mean excessive use of Local Memory might slow down kernels on CPUs
 - GPUs now have effective on-chip caches which can provide much of the benefit of Local Memory but without programmer intervention
 - So, your mileage may vary!

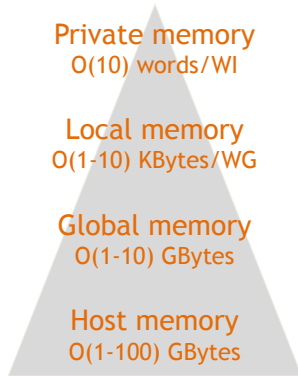


The Memory Hierarchy

Bandwidths



Sizes



Speeds and feeds approx. for a high-end discrete GPU, circa 2011

Memory Consistency

- OpenCL uses a **relaxed consistency** memory model; i.e.
 - The state of memory visible to a work-item is not guaranteed to be consistent across the collection of work-items at all times.
- Within a work-item:
 - Memory has load/store consistency to the work-item's private view of memory, i.e. it sees its own reads and writes correctly
- Within a work-group:
 - Local memory is consistent between work-items at a barrier.
- Global memory is consistent within a work-group at a barrier, **but *not* guaranteed across different work-groups!!**
 - This is a common source of bugs!
- Consistency of memory shared between **commands** (e.g. kernel invocations) is enforced by **synchronization** (barriers, events, in-order queue)

Optimizing matrix multiplication

- There may be significant overhead to manage work-items and work-groups.
- So let's have each work-item compute a full row of C

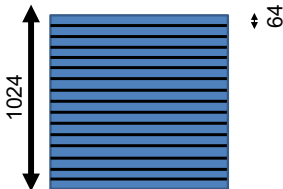
The diagram shows the calculation of a single element $C(i,j)$ in matrix C. It is represented as the dot product of a row $A(i,:)$ from matrix A and a column $B(:,j)$ from matrix B. Matrix A is shown with a red horizontal band representing row i . Matrix B is shown with a red vertical band representing column j . The result $C(i,j)$ is shown as a single element in a red band of matrix C.

Dot product of a row of A and a column of B for each element of C

- And with an eye towards future optimizations, let's collect work-items into work-groups with 64 work-items per work-group

An N-dimension domain of work-items

- **Global** Dimensions: 1024 (1D)
Whole problem space (index space)
- **Local** Dimensions: 64 (work-items per work-group)
Only $1024/64 = 16$ work-groups in total



- Important implication: we will have a lot fewer work-items per work-group (64) and work-groups (16). Why might this matter?

Matrix multiplication: One work item per row of C

```
__kernel void mmul(  
    const int N,  
    __global float *A,  
    __global float *B,  
    __global float *C)  
{  
    int j, k;  
    int i = get_global_id(0);  
    float tmp;  
    for (j = 0; j < N; j++) {  
        tmp = 0.0f;  
        for (k = 0; k < N; k++)  
            tmp += A[i*N+k]*B[k*N+j];  
        C[i*N+j] = tmp;  
    }  
}
```

Matrix multiplication host program (C++ API)

Changes to host program:

1. 1D ND Range set to number of rows in the C matrix
2. Local Dimension set to 64 so number of work-groups match number of compute units (16 in this case) for our order 1024 matrices

```
int main(int argc, char** argv)
{
    std::vector<int> szA, szB, szC;
    int Mdim, Ndim, Pdim;
    int i, err;
    int szA, szB, szC;
    double start_time, run_time;
    cl::Program program;

    Ndim = Pdim = Mdim = ORDER;
    szA = Ndim*Mdim;
    szB = Pdim*Mdim;
    szC = Ndim*Mdim;
    h_A = std::vector<float>(szA);
    h_B = std::vector<float>(szB);
    h_C = std::vector<float>(szC);

    initmat(Mdim, Ndim, Pdim, h_A, h_B, h_C);

    // Compile for first kernel to setup program
    program = cl::Program(C_elem_KernelSource, true);
    Context context(CL_DEVICE_TYPE_DEFAULT);
    cl::CommandQueue queue(context);
    std::vector<Device> devices =
        context.getInfo<CL_CONTEXT_DEVICES>();
    cl::Device device = devices[0];
    std::string s =
        device.getInfo<CL_DEVICE_NAME>();
    std::cout << "\nUsing OpenCL Device "
        << s << "\n";

    cl::make_kernel<int, int, int>
        krow(cl::Buffer, cl::Buffer, cl::Buffer>
            krow(program, "mmul");

    zero_mat(Ndim, Mdim, h_C);
    start_time = wtime();

    krow(cl::EnqueueArgs(queue,
        cl::NDRange(Ndim),
        cl::NDRange(ORDER/16)),
        Ndim, Mdim, Pdim, a_in, b_in, c_out);

    cl::copy(queue, d_c, h_C.begin(), h_C.end());

    run_time = wtime() - start_time;
    results(Mdim, Ndim, Pdim, h_C, run_time);
}
```

Matrix multiplication performance

- Matrices are stored in global memory.

Case	MFLOPS	
	CPU	GPU
Sequential C (not OpenCL)	887.2	N/A
C(i,j) per work-item, all global	3,926.1	3,720.9
C row per work-item, all global	3,379.5	4,195.8

This has started to help. 

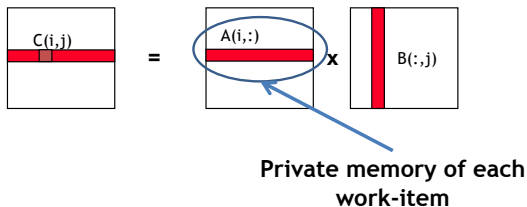
Device is Tesla® M2090 GPU from NVIDIA® with a max of 16 compute units, 512 PEs
Device is Intel® Xeon® CPU, E5649 @ 2.53GHz

These are not official benchmark results. You may observe completely different results should you run these tests on your own system.

Third party names are the property of their owners.

Optimizing matrix multiplication

- Notice that, in one row of C , each element reuses the same row of A .
- Let's copy that row of A into private memory of the work-item that's (exclusively) using it to avoid the overhead of loading it from global memory for each $C(i,j)$ computation.



Matrix multiplication: (Row of A in private memory)

Copy a row of A into private memory from global memory before we start with the matrix multiplications.

```
__kernel void mmul(  
    const int N,  
    __global float *A,  
    __global float *B,  
    __global float *C)  
{  
    int j, k;  
    int i =  
        get_global_id(0);  
    float tmp;  
    float Awrk[1024];
```

Setup a work array for A in private memory*

```
    for (k = 0; k < N; k++)  
        Awrk[k] = A[i*N+k];  
  
    for (j = 0; j < N; j++) {  
        tmp = 0.0f;  
        for (k = 0; k < N; k++)  
            tmp += Awrk[k]*B[k*N+j];  
  
        C[i*N+j] += tmp;  
    }  
}
```

(*Actually, this is using *far* more private memory than we'll have and so Awrk[] will be spilled to global memory)

Matrix multiplication performance

- Matrices are stored in global memory.

Case	MFLOPS	
	CPU	GPU
Sequential C (not OpenCL)	887.2	N/A
C(i,j) per work-item, all global	3,926.1	3,720.9
C row per work-item, all global	3,379.5	4,195.8
C row per work-item, A row private	3,385.8	8,584.3

Device is Tesla® M2090 GPU from
NVIDIA® with a max of 16
compute units, 512 PEs
Device is Intel® Xeon® CPU,
E5649 @ 2.53GHz

Big impact!



These are not official benchmark results. You may observe completely different results should you run these tests on your own system.

Third party names are the property of their owners.

Why using too much private memory can be a good thing

- In reality private memory is just hardware registers, so only dozens of these are available per work-item
- Many kernels will allocate too many variables to private memory
- So the compiler already has to be able to deal with this
- It does so by *spilling* excess private variables to (global) memory
- You still told the compiler something useful - that the data will only be accessed by a single work-item
- This lets the compiler allocate the data in such a way as to enable more efficient memory access



Memory Hierarchy

Exercise 6: Matrix Multiplication (~20 minutes)

■ Goal:

Use private memory to minimize memory movement costs and optimize performance of your matrix multiplication program.

■ Procedure:

Start with the provided matrix multiplication OpenCL code. Complete the kernels `C_row.cl`: each work-item compute a full row of `C`, and `C_row_priv.cl`: row per work-item. A row private. In addition, complete the blank parts in the host code `matmul` to run both kernels.

■ Output:

A message verifying the code has completed successfully. Report the runtime and the MFLOPS.



```
...
}

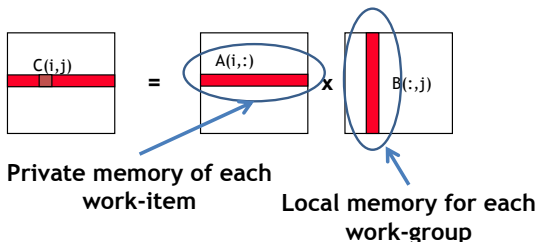
__kernel void C_row(__global const float *A, __global const float *B, __global float *C,
                    const int rows, const int cols, const int rows_per_workitem) {
    const int row_id = get_global_id(0);
    const int row_stride = rows_per_workitem;
    const int row_start = row_id * row_stride;
    const int row_end = row_start + row_stride;

    for (int row = row_start; row < row_end; row++) {
        for (int col = 0; col < cols; col++) {
            float sum = 0.0f;
            for (int k = 0; k < rows; k++) {
                sum += A[row + k * rows] * B[k * cols + col];
            }
            C[row * cols + col] = sum;
        }
    }
}

int main() {
    // Host code to run the kernel
    // ...
}
```

Optimizing matrix multiplication

- We already noticed that, in one row of C , each element uses the same row of A
- Each work-item in a work-group also uses the same columns of B
- So let's store the B columns in **local** memory (which is shared by the work-items in the work-group)



Matrix multiplication: B column shared between work-items

```
__kernel void mmul(  
    const int N,  
    __global float *A,  
    __global float *B,  
    __global float *C,  
    local float *Bwrk)  
{  
    int j, k;  
    int i =  
        get_global_id(0);  
  
    int iloc =  
        get_local_id(0);  
  
    int nloc =  
        get_local_size(0);  
  
    float tmp;  
    float Awrk[1024];  
}
```

```
    for (k = 0; k < N; k++)  
        Awrk[k] = A[i*N+k];  
  
    for (j = 0; j < N; j++) {  
        for (k=iloc; k<N; k+=nloc)  
            Bwrk[k] = B[k* N+j];  
  
        barrier(CLK_LOCAL_MEM_FENCE);  
  
        tmp = 0.0f;  
        for (k = 0; k < N; k++)  
            tmp += Awrk[k]*Bwrk[k];  
  
        C[i*N+j] = tmp;  
  
        barrier(CLK_LOCAL_MEM_FENCE);  
    }  
}
```

Pass a work array in local memory to hold a column of B. All the work-items do the copy "in parallel" using a cyclic loop distribution (hence why we need iloc and nloc)

Matrix multiplication host program (C++ API)

Changes to host program:

1. Pass local memory to kernels.
 1. This requires a change to the kernel argument lists ... an arg of type LocalSpaceArg is needed.
2. Allocate the size of local memory
3. Update argument list in kernel functor

```
nd(), true);
nd(), true);

ONLY,
* szC);
```

```
Ndim = Pdim = Mdim = ORDER;
szA = Ndim*Pdim;
szB = Pdim*Mdim;
szC = Ndim*Mdim;
h_A = std::vector<float>(szA);
h_B = std::vector<float>(szB);
h_C = std::vector<float>(szC);

initmat(Mdim, Ndim, Pdim, h_A, h_B, h_C);

// Compile for first kernel to setup program
program = cl::Program(C_elem_KernelSource, true);
Context context(CL_DEVICE_TYPE_DEFAULT);
cl::CommandQueue queue(context);
std::vector<Device> devices =
    context.getInfo<CL_CONTEXT_DEVICES>();
cl::Device device = devices[0];
std::string s =
    device.getInfo<CL_DEVICE_NAME>();
std::cout << "\nUsing OpenCL Device "
    << s << "\n";
```

```
cl::LocalSpaceArg localmem =
    cl::Local(sizeof(float) * Pdim);
```

```
cl::make_kernel<int, int, int,
    cl::Buffer, cl::Buffer, cl::Buffer,
    cl::LocalSpaceArg>
    rowcol(program, "mmul");
```

```
zero_mat(Ndim, Mdim, h_C);
start_time = wtime();
```

```
rowcol(cl::EnqueueArgs(queue,
    cl::NDRange(Ndim),
    cl::NDRange(ORDER/16)),
    Ndim, Mdim, Pdim, d_a, d_b, d_c, localmem);
```

```
cl::copy(queue, d_c, h_C.begin(), h_C.end());
```

```
run_time = wtime() - start_time;
results(Mdim, Ndim, Pdim, h_C, run_time);
}
```


Matrix multiplication performance

- Matrices are stored in global memory.

Case	MFLOPS	
	CPU	GPU
Sequential C (not OpenCL)	887.2	N/A
C(i,j) per work-item, all global	3,926.1	3,720.9
C row per work-item, all global	3,379.5	4,195.8
C row per work-item, A row private	3,385.8	8,584.3
C row per work-item, A private, B local	10,047.5	8,181.9

Device is Tesla® M2090 GPU from NVIDIA® with a max of 16 compute units, 512 PEs
Device is Intel® Xeon® CPU, E5649 @ 2.53GHz

These are not official benchmark results. You may observe completely different results should you run these tests on your own system.

Third party names are the property of their owners.

Making matrix multiplication *really* fast

- Our goal has been to describe how to work with private, local and global memory. We've ignored many well-known techniques for making matrix multiplication fast
 - The number of work items must be a multiple of the fundamental machine “vector width”. This is the wavefront on AMD, warp on NVIDIA, and the number of SIMD lanes exposed by vector units on a CPU
 - To optimize reuse of data, you need to use blocking techniques
 - Decompose matrices into tiles such that three tiles just fit in the fastest (private) memory
 - Copy tiles into local memory
 - Do the multiplication over the tiles
 - We modified the matrix multiplication program provided with the NVIDIA OpenCL SDK to work with our test suite to produce the blocked results on the following slide. This used register blocking with block sizes mapped onto the GPU's warp size

Matrix multiplication performance

- Matrices are stored in global memory.

Case	MFLOPS	
	CPU	GPU
Sequential C (not OpenCL)	887.2	N/A
C(i,j) per work-item, all global	3,926.1	3,720.9
C row per work-item, all global	3,379.5	4,195.8
C row per work-item, A row private	3,385.8	8,584.3
C row per work-item, A private, B local	10,047.5	8,181.9
Block oriented approach using local	1,534.0	230,416.7

Device is Tesla® M2090 GPU from
NVIDIA® with a max of 16
compute units, 512 PEs
Device is Intel® Xeon® CPU,
E5649 @ 2.53GHz

Third party names are the property of their owners.

Biggest impact so far!



These are not official benchmark results. You may observe completely different results should you run these tests on your own system.

Memory Hierarchy

Exercise 7: Matrix Multiplication (~10 minutes)

■ Goal:

Use local memory to minimize memory movement costs and optimize performance of your matrix multiplication program

■ Procedure:

Start with the provided matrix multiplication OpenCL code. Complete the kernel `C_row_priv_bloc.cl`: row per work-item, A private, B local. In addition, complete the blank parts in the host code `matmul` to run the kernel.

■ Output:

A message verifying the code has completed successfully. Report the runtime and the MFLOPS.

■ Extra:

Look at the fast, blocked implementation from the NVIDIA OpenCL SDK example. Try running it and compare to yours



```

// OpenCL C code for matrix multiplication kernel
// Kernel: C_row_priv_bloc
// Inputs: 3D buffers A, B, C and 2D index (i, j)
// Outputs: 3D buffer C

// Local memory variables
__local float rowA[128];
__local float colB[128];

// Global memory variables
__global float *A, *B, *C;
__global int *i, *j;

// Kernel execution
__kernel void C_row_priv_bloc(__global float *A, __global float *B, __global float *C, __global int *i, __global int *j) {
    // Get local and global IDs
    int lid = get_local_id(0);
    int gid = get_global_id(0);

    // Calculate row and column indices
    int rowA_idx = gid * 128;
    int colB_idx = gid * 128;

    // Load row A into local memory
    for (int k = 0; k < 128; k++) {
        rowA[k] = A[rowA_idx + k];
    }

    // Load column B into local memory
    for (int k = 0; k < 128; k++) {
        colB[k] = B[colB_idx + k];
    }

    // Calculate dot product
    float result = 0.0f;
    for (int k = 0; k < 128; k++) {
        result += rowA[k] * colB[k];
    }

    // Store result in C
    C[i[gid] * 128 + j[gid]] = result;
}

```

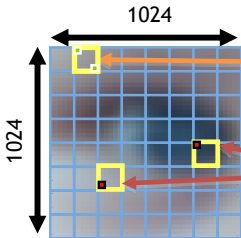
Table of contents

- 1 An Overview of OpenCL
- 2 Important OpenCL concepts
- 3 Overview OpenCL APIs
- 4 Kernel Programming
- 5 Memory Hierarchy
- 6 Synchronization**
- 7 Heterogeneous Computing



Consider N-dimensional domain of work-items

- **Global** Dimensions:
 - 1024x1024 (whole problem space)
- **Local** Dimensions:
 - 64x64 (**work-group**, executes together)



Synchronization between **work-items** possible only within **work-groups**:
barriers and **memory fences**

Cannot synchronize between **work-groups** within a kernel

Synchronization: when multiple units of execution (e.g. work-items) are brought to a known point in their execution. Most common example is a barrier ... i.e. all units of execution “in scope” arrive at the **barrier** before any proceed.

Work-Item Synchronization

Ensure correct order of memory operations to local or global memory (with flushes or queuing a memory fence)



- Within a work-group

void barrier()

- Takes optional flags

CLK_LOCAL_MEM_FENCE and/or **CLK_GLOBAL_MEM_FENCE**

- A work-item that encounters a **barrier()** will wait until ALL work-items in its work-group reach the **barrier()**
- **Corollary:** If a **barrier()** is inside a branch, then the branch **must** be taken by either:
 - **ALL** work-items in the work-group, OR
 - **NO** work-item in the work-group

- Across work-groups

- No guarantees as to where and when a particular work-group will be executed relative to another work-group
- **Cannot exchange data, or have barrier-like synchronization between two different work-groups! (Critical issue!)**
- **Only solution: finish the kernel and start another**

Where might we need synchronization?

- Consider a reduction ... reduce a set of numbers to a single value
 - E.g. find sum of all elements in an array
- Sequential code

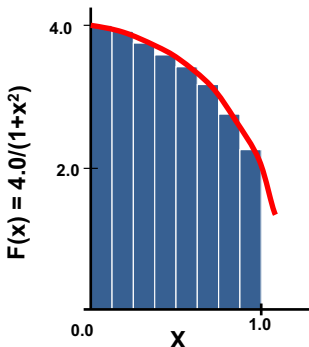
```
int reduce(int Ndim, int *A)
{
    int sum = 0;
    for (int i = 0; i < Ndim; i++)
        sum += A[i];
    return sum;
}
```


Simple parallel reduction

- A reduction can be carried out in three steps:
 1. Each work-item sums its private values into a local array indexed by the work-item's local id
 2. When all the work-items have finished, one work-item sums the local array into an element of a global array (indexed by work-group id).
 3. When all work-groups have finished the kernel execution, the global array is summed on the host.
- Note: this is a simple reduction that is straightforward to implement. More efficient reductions do the work-group sums in parallel on the device rather than on the host. These more scalable reductions are considerably more complicated to implement.

A simple program that uses a reduction

Numerical Integration



Mathematically, we know that we can approximate the integral as a sum of rectangles.

Each rectangle has width and height at the middle of interval.

Numerical integration source code

The serial Pi program

```
static long num_steps = 100000;
double step;
void main()
{
    int i; double x, pi, sum = 0.0;

    step = 1.0/(double) num_steps;

    for (i = 0; i < num_steps; i++) {
        x = (i+0.5)*step;
        sum = sum + 4.0/(1.0+x*x);
    }
    pi = step * sum;
}
```

Synchronization

Exercise 8: The Pi program Homework

■ Goal:

To understand synchronization between work-items in the OpenCL C kernel programming language.

■ Procedure:

Start with the provided serial program to estimate Pi. Write a kernel and host program to compute the numerical integral using OpenCL. **Note: You will need to implement a reduction.**

■ Output:

Output result plus an estimate of the error in the result. Report the runtime.



Hint: you will want each work-item to do many iterations of the loop, i.e. don't create one work-item per loop iteration. To do so would make the reduction so costly that performance would be terrible.

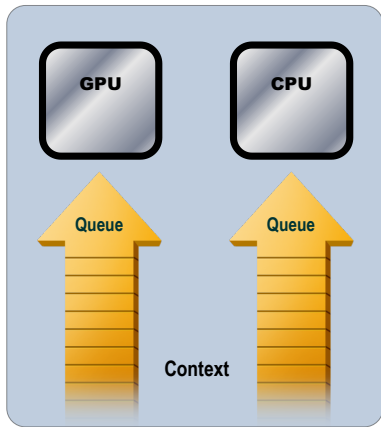
Table of contents

- 1 An Overview of OpenCL
- 2 Important OpenCL concepts
- 3 Overview OpenCL APIs
- 4 Kernel Programming
- 5 Memory Hierarchy
- 6 Synchronization
- 7 Heterogeneous Computing



Running on the CPU and GPU

- Kernels can be run on multiple devices at the same time
- We can exploit many GPUs and the host CPU for computation
- Simply define a context with multiple platforms, devices and queues
- We can even synchronize between queues using Events (see appendix)
- Can have more than one context



Running on the CPU and GPU

1. Discover all your platforms and devices
 - Look at the API for finding out Platform and Device IDs
2. Set up the `cl::Context` with a vector of devices

```
cl::Context(const VECTOR_CLASS<Device> &devices,  
            cl_context_properties *properties = NULL,  
            void (CL_CALLBACK *pfn_notify) (  
                const char *errorinfo,  
                const void *private_info_size,  
                ::size_t cb, void *user_data) = NULL,  
            void *user_data = NULL, cl_int *err = NULL);
```

3. Create a Command Queue for each of these devices
 - C examples in the NVIDIA (oclSimpleMultiGPU) and AMD (SimpleMultiDevice) OpenCL SDKs

The steps are the same in C and Python, just the API calls differ as usual

Heterogeneous Computing

Exercise 9: Heterogeneous Computing Homework

- **Goal:**

To experiment with running kernels CONCURRENTLY on multiple devices.

- **Procedure:**

Take the vadd OpenCL program (Exercise 2).

Investigate the Context constructors to include more than one device. Modify the program to run a kernel concurrently on multiple devices, each with different input data. Split your problem across multiple devices.

- **Output:**

A message verifying that the vector addition completed successfully. See which device runs faster.

